Trading Bot Convergence Framework — Corrected v2

Purpose

A convergence optimization engine for a Solana micro-cap trading bot. The system drives to zero **objective defects** through systematic code analysis, then validates with empirical data.

Core Principle (Corrected)

The bot is a **deterministic algorithm operating on stochastic inputs**. You can PROVE what the code will compute given assumed inputs, but you cannot PROVE market outcomes. Code-correctness is provable from source alone; behavioral prediction requires empirical validation.

Phase 0 — Define the Objective

```
GOAL:              Maximize expected net profit per trade aft
er ALL costs.

DECISION VARIABLE:  x = enter token T at size s (or skip)

f(x) = captureMult(score) * E[peak_move | liq] - round_trip_c
ost(liq)
  captureMult(score) = 0.7 + 1.3 * clamp((score - 70)/20, 0,
1.2)
  round_trip_cost(liq) = entry_slip + exit_slip + fees
    // ~6.8% @ $25k liquidity ... ~15.5% @ $5k liquidity
```

```
DIRECTION:          maximize f(x)
DECISION RULE:      ENTER iff f(x) > +1.0% (OBJECTIVE_EV_MARGIN); else SKIP
CONVERGENCE CHECK:  realized mean(f(x)) > +1.0% over >= 30 trades (paper or live)
LOSS FUNCTION:      count of OBJECTIVE defects = paths that are code-correct but do NOT
                    provably raise E[f(x)] given validated inputs.

INPUT CLASSIFICATION (must be labeled for every path):
  score             -> descriptive; may inform sizing but NEVER standalone entry trigger
  recent momentum   -> descriptive; may inform sizing but NEVER standalone entry trigger
  estimated peak    -> descriptive prior (estimateRealisticPeakPct); NOT a forecast
  liquidity         -> structural/real; used for cost feasibility and sizing only
  forward signal    -> PREDICTIVE (if any); may trigger entry if forward-validated

RULE: If NO predictive input exists, E[capture] ≈ 0. The bot must:
  (a) acquire and validate a predictive signal, OR
  (b) report objective-infeasible and halt trading.
```

The Loop

```
Phase 0 (OBJECTIVE) → Phase A (SCAN) → Phase B (FIX) → Phase C (RE-SCAN) → Phase D (CHECK)
     ↑
|
```

```
      └──────────────────────────────── if defects found ──────────────
──────────────────────────┘
```

After Phase D (analytical convergence), proceed to:

Phase E (EMPIRICAL VALIDATION): Paper-trade for >= 30 trades. If realized mean(f(x)) < 1.0%, return to Phase 0 — the inputs or model are defective, not the code.

Phase A — SCAN (Comprehensive, No Fixes)

A1. Enumerate every code path:

- Scoring path (score computation, bonuses, penalties)
- Entry gates (minScoreToTrade, sniperMinScore, sniperMinLiquidity, mgMinLiquidity, mgMinScore, QUALITY_LIQ_FLOOR, EDGE_POCKET_ONLY)
- Objective-EV gate (objectiveNetEvPct, OBJECTIVE_EV_MARGIN)
- Cost model (entry_slip, exit_slip, fees, calcCostAwareStopPrice)
- Exit paths (stop -5%, trailing arm +8%, partial-TP 0.5 @ +3%, cooldowns)
- Zero-liquidity guard and edge-pocket path
- All interaction points where one subsystem feeds another

A2. Trace each path with concrete values. Example:

> liq = $10k → round_trip_cost ≈ 9.75%. Token must peak > +9.75% just to break even.

> Most tokens in this class peak +3–8%. → UN-WINNABLE. The arithmetic is the proof.

> Candidate: score 87, liq $13k. round_trip_cost ≈ 8.6%. estimateRealisticPeakPct(13k) = 5%.

> f(x) = captureMult(87) *5% - 8.6% = 1.805* 5% - 8.6% = 9.025% - 8.6% = +0.425%.

> f(x) < 1.0% → gate SKIPS. Code-correct AND objective-correct: OK.

> Note: estimated peak is a descriptive prior, not a forecast. If used as expected realized capture, that is a PREDICTIVE-VALIDITY DEFECT.

A3. Classify each finding:

Classification	Question	Action
DEFECT	Value moves the objective in the wrong direction?	Record. Do NOT fix yet.
DEAD CODE	Computed but never used?	Record. Do NOT fix yet.
INTERACTION	Two parameters create an impossible combination?	Record. Do NOT fix yet.
CONTRADICTION	Comment/config says one thing, value another?	Record. Do NOT fix yet.
UNREACHABLE	Threshold above any reachable value?	Record. Do NOT fix yet.
UN-WINNABLE	Round-trip cost exceeds achievable move?	Record. Do NOT fix yet.
OBJECTIVE DEFECT	Code-correct but fails to raise E[f(x)]?	Record. Do NOT fix yet.
PREDICTIVE-VALIDITY DEFECT	Past-describing signal used as forecast without proof?	Record. Do NOT fix yet.
DEFECTIVE APPROACH	Proposed change violates objective or constraint?	Record. Do NOT construct.
VERIFIED OK	Path works AND advances the objective?	Mark verified. No action.

A4. Record ALL findings before touching code: ID (D1, D2...), severity, file:line + quoted code, current vs correct, and arithmetic proof.

A5. Interaction check: For every pair that feeds each other, ask: "Can a valid candidate pass ALL gates in sequence and still have f(x) > margin?" A score bonus that lifts a token past a gate whose downstream cost makes f(x) negative is a joint defect even if each part is correct.

Phase B — FIX

Apply ALL recorded fixes in one pass. For each:

- Add a comment explaining what was wrong, why the new value is correct, and the arithmetic.
- Re-read surrounding lines for immediate interactions.
- Do NOT build a fix on an unvalidated descriptive prior.
- After fixing, verify the code still compiles/transpiles.

Phase C — RE-SCAN

Re-run the ENTIRE Phase A scan against the fixed code. Re-trace each path, re-check every interaction, confirm no fix created a new defect and none was too aggressive (blocking legitimate profitable entries is itself a defect). New defects → back to Phase A.

Phase D — CONVERGENCE CHECK

> Did the last full A + C scan find ZERO new defects across ALL representative paths AND all interaction checks pass?

If YES → **analytically converged**. Proceed to Phase E (empirical validation).

If NO → back to Phase A.

Analytical convergence is NOT reality convergence. It proves code-correctness and objective alignment for the assumed model. Real-world performance is tested separately.

Phase E — EMPIRICAL VALIDATION

Rule: Deploy to **PAPER ONLY** after Phase D. Live trading only after paper validation.

Acceptance criterion: Realized mean(f(x)) > +1.0% over >= 30 trades.

If paper fails:

- The code has zero defects (proven in Phase D)
- Therefore the defect is in the **inputs or model** (descriptive signals treated as predictive, wrong cost estimates, incorrect peak priors, etc.)
- Return to Phase 0. Re-examine input classification, cost model, or peak estimates.
- Do NOT add more gates to "fix" a broken model.

ANTI-DRIFT RULES (Corrected)

R1. Follow phases in order. Never skip Phase 0 or A. Never fix during scan. Never deploy before Phase D.

R2. No invented names. Phases 0/A/B/C/D/E. Defects D1/D2/D3.

R3. Code is primary evidence. Read code to find defects. Use logs/trades only to confirm or classify known defects, not to discover them from scratch.

R4. Trace to source. A bad exit is often born at entry or scoring. Walk upstream.

R5. Check interactions. "Can a valid candidate clear ALL gates AND still have f(x) > margin?"

R6. Record everything. Every defect, fix, and re-scan result. A stale record is drift.

R7. No premature convergence. Zero defects across ALL representative paths + ALL interactions, nothing less.

R8. Do not react to one trade. One win or loss is not a pattern; judge over >= 30 trades.

R9. Trace upstream. A bad exit is often born at entry or scoring. Walk upstream.

R10. Do not overshoot. A gate that blocks profitable entries is worse than no gate.

R11. Converge on the objective, not the code. Zero bugs is necessary, not sufficient. Trace every path for code-correctness AND for raising E[f(x)].

R12. Descriptive != predictive. Score, momentum, and estimated peak describe the past. They may inform f(x) but may NOT trigger an entry alone until forward

validity is proven. This applies to fixes too — do not build fixes on unvalidated priors.

R13. Analytical convergence != empirical success. Phase D proves code. Phase E proves reality. Both are required.

R14. If NO predictive input exists, the bot is infeasible. Do not code around this. Fix the signal acquisition or halt.

OUTPUT FORMAT FOR EACH FINDING

```
ID: D{N}
MODE: VERIFY / SOLVE
SEVERITY: CRITICAL / MEDIUM / LOW
LOCATION: {file}:{line} — "{quoted code}"
CURRENT VALUE/APPROACH: {what it is}
CORRECT VALUE/APPROACH: {what it should be}
CLASSIFICATION: [see table above]
ARITHMETIC PROOF: {hand-trace with concrete liq/score/cost va
lues}
OBJECTIVE IMPACT: {how this changes E[f(x)]}
FIX: {exact code change}
BLAST RADIUS: {what else this affects; what was NOT touched}
INTERACTION CHECK: {does this interact with other fixes/gate
s?}
VERIFICATION: {how to confirm from code}
STILL UNVERIFIED: {what needs empirical validation}
```

BUG-CLASS CHECKLIST

- [ ] Symptom-vs-source: bad exit born at scoring/entry
- [ ] Computed-but-not-enforced: value calculated, never gated on
- [ ] Unreachable trigger: threshold above any reachable score/price
- [ ] Reactive floor gapped: price jumps a stop between checks

- [ ] Sunk-cost double-count: re-charging already-paid entry cost in the stop
- [ ] Un-winnable geometry: round-trip cost exceeds achievable peak
- [ ] Boundary errors: < vs <=, > vs >= on score/liq/price gates
- [ ] Two paths that look like one: guard on sniper path, action on momentum path
- [ ] Misleading diagnostics: stale labels causing wrong conclusions
- [ ] Objective defect: gate is code-correct but does not raise E[f(x)]
- [ ] Predictive-validity defect: a gate/fix uses a past-describing signal as a forecast
- [ ] Interaction impossibility: score bonus lifts token past a gate whose cost makes f(x) < 0
- [ ] Defective approach: a proposed change violates the objective or a constraint
- [ ] Double-counting: same variable applied twice in a formula (e.g., conviction * captureMult where captureMult already includes conviction)

KNOWN DEFECTS ON RECORD (Updated)

- **OD-1 PREDICTIVE-VALIDITY (OPEN):** Entry inputs are lagging/descriptive; no proven forward signal. Theoretical basis: descriptive statistics have no inherent forward validity. Remediation: Acquire and validate a predictive signal, or report infeasibility. Closing requires forward-validation of a signal against >= 30 out-of-sample trades.
- **OD-2 UN-WINNABLE LIQUIDITY (FIXED):** Entries where cost > achievable move; closed by the objective-EV gate + liquidity floors. Verified analytically (rejects score 80–87 at ~$13–14k liq). Empirically confirmed on paper trades.
- **OD-3 PARTIAL-TP UNREACHABLE AT LOW LIQ (WATCH):** +3% partial-TP band can sit inside the cost spread at low liquidity; verify reachability per liq tier before each deployment.
- **OD-4 SCORE NOT EV-VALIDATED (WATCH):** Score must remain an INPUT to f(x), never the decision trigger. Entry must be driven by f(x) > margin, not score > threshold.

- **OD-5 CONVICTION DOUBLE-COUNTING (FIXED):** Original f(x) multiplied conviction by captureMult, but captureMult already included conviction. Fixed by removing explicit conviction factor from f(x) and using captureMult alone.

SUMMARY

```
Phase 0: Define f(x) and label every input descriptive/predic
tive. Trace for code AND objective.
Phase A: Scan ALL representative paths. Record ALL findings.
Do NOT fix.
Phase B: Fix ALL defects. One pass. No fix built on an unvali
dated prior.
Phase C: Re-scan ALL paths. Check interactions.
Phase D: Zero defects -> analytically converged -> proceed to
Phase E.
Phase E: Paper-trade. Realized mean(f(x)) > +1.0% over >= 30
trades.
If paper fails: return to Phase 0. The model/inputs are wron
g, not the code.
```

Follow this. Do not drift. Execute the loop.